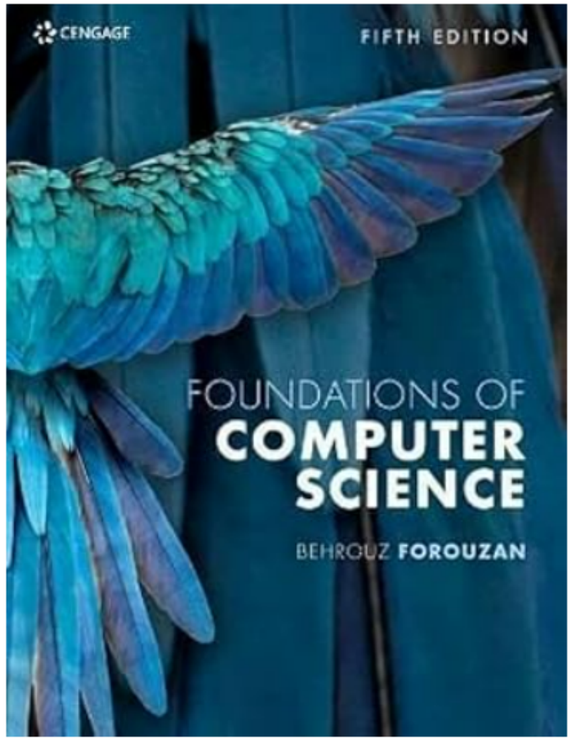
2026 Spring

Course ID: 1142-1764

Foundations of Computer Science

10-Networking and Content Delivery

The Textbook



Foundations of Computer Science Paperback

作者 Behrouz Forouzan (Author)

The symposium was held Oct.-Nov. 1989, Research Triangle Park, North Carolina. One hundred papers in theoretical computer science treat dispersers, deterministic amplification, and weak random sources; efficient NC algorithms for set cover with applications to learning and geometry; the inverse of automorphism in polynomial time; and speeding-up linear programming using fast matrix multiplication. Acidic paper; no subject index. Annotation copyright Book News, Inc. Portland, Or.

報告此產品或賣家的問題

語言	尺寸	ISBN-10	ISBN-13
English	18.9 x 24.7 x 2.6 cm	1473787327	978-1473787322

查看所有詳細內容

AWS ACADEMY · CLOUD FOUNDATIONS

Flip Classroom

Padlet Collaborative Learning

Introduction to Computer Science | Modules 5 – 10

- M5 Networking
- M6 Compute
- M7 Storage
- M8 Databases
- M9 Architecture
- M10 Auto Scaling

📌 *Your Group Builds the Knowledge Base — Together*

Grading

The report's grading includes:

- 20% teacher's score
- 20% group score
- 60% peer evaluation.

Group leaders:
Module 6 - 411425209 LE TUAN ANH黎俊英
Module 7 - 411416878 CALVIN THEODORIC LAVENCIO 鄭鎧文
Module 8 - 411416886 MANDY JANNYCIA MAHIDIN 蔡美彦
Module 9 - 411416991 Yansen Ladova 許原盛
Module 10 - E11420600 UDAYA SANKAR HARIHARAN哈里哈蘭

What Is a Flip Classroom?

1 BEFORE Class

Study the module
watch videos,
read materials at home

2 PREPARE Padlet

Your group fills in
all 5 sections of
the Padlet template

3 IN Class

Present & discuss
answer classmates'
questions (Q&A)

4 REFLECT

Respond to peer
comments, update
the Padlet

💡 Padlet is the shared workspace where your group documents your learning for each module — visible to the whole class!

Your Padlet Template — 5 Sections

S1

Learning Objectives

State the key learning goals of the module.
What should you understand by the end?

S2

Core Concepts

Summarize essential technical concepts,
terminology, and architectures.

S3

Labs

Document hands-on AWS lab steps,
screenshots, and observations.

S4

Challenges

Record difficult questions, errors
you encountered, and how you solved them.

S5

Q&A & Peer Support

Post questions for the class.
Answer and support your peers.

Tips for a Great Padlet

Add images & diagrams, not just text

Link to AWS documentation & tutorials

Use clear headings in each column

Respond to at least 2 peers in S5

M5

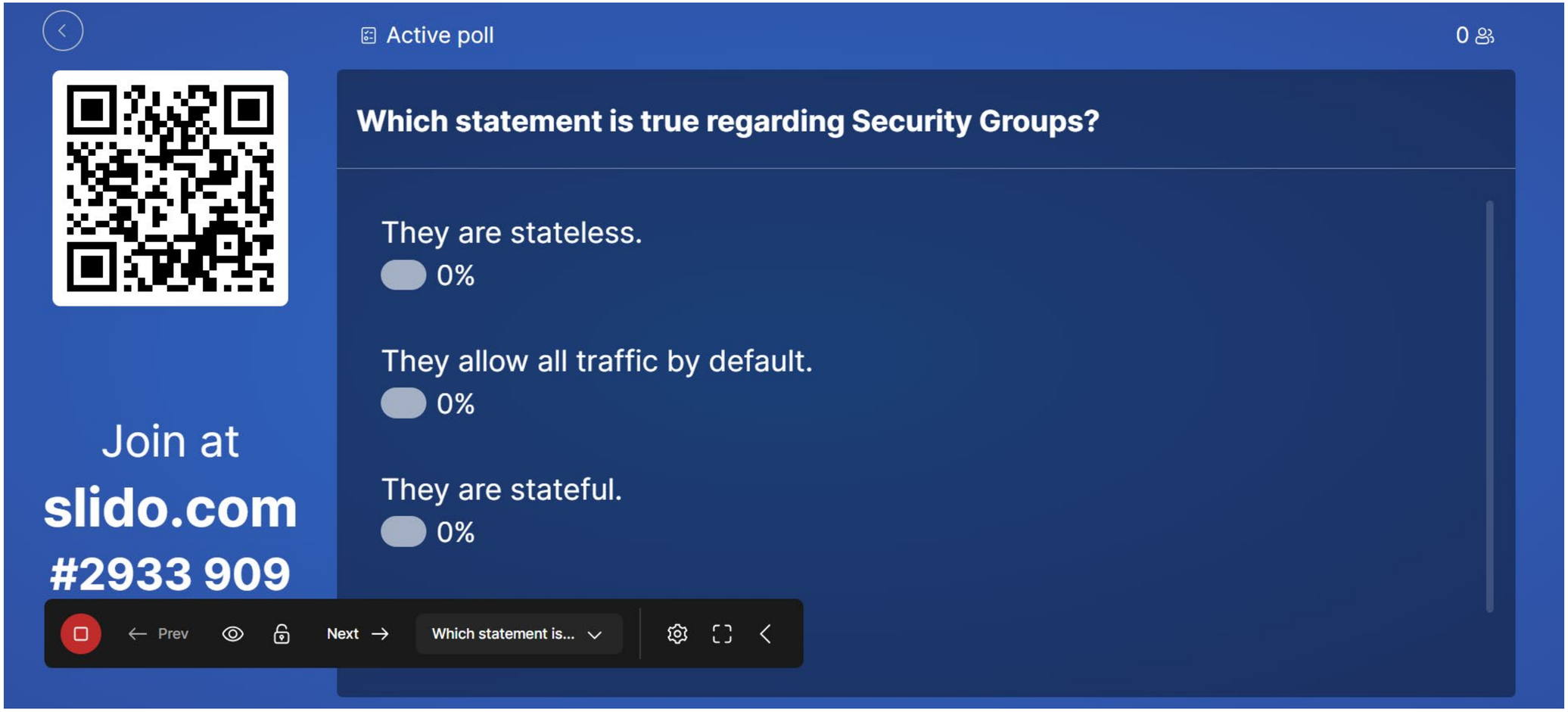
Networking & Content Delivery

📘 Core Concepts to Explore

- VPC, Subnets & CIDR Blocks
- Internet Gateway & Route Tables
- Security Groups vs. NACLs
- Amazon CloudFront CDN
- Elastic Load Balancing (ELB)

📌 Your Padlet Tasks

S1	**Learning Objectives**	List 3–5 objectives for this module
S2	**Core Concepts**	Diagram a VPC with public & private subnets
S3	**Lab Summary**	AWS Lab – Build a VPC; screenshot your architecture
S4	**Challenges**	Why use private subnets? What confused you about NACLs?
S5	**Q&A**	Post 1 question, reply to 2 peers


Active poll
0
Join at
slido.com
#2933 909
Which statement is true regarding Security Groups?
They are stateless.
0%
They allow all traffic by default.
0%
They are stateful.
0%
Prev
Next
Which statement is...

M6 Compute

📘 Core Concepts to Explore

- EC2 Instance Types & AMIs
- Pricing Models (On-Demand, Reserved, Spot)
- Elastic Beanstalk & Lambda
- Auto Scaling Groups Intro
- AWS Pricing Calculator

📌 Your Padlet Tasks

S1	**Learning Objectives**	List 3–5 objectives for this module
S2	**Core Concepts**	Compare EC2 pricing models in a table
S3	**Lab Summary**	Launch & connect to an EC2 instance; log steps
S4	**Challenges**	What's the trade-off between Reserved vs. Spot instances?
S5	**Q&A**	Post 1 question, reply to 2 peers

AWS Academy Cloud Foundations

Module 6

Compute

Amazon EC2 · Auto Scaling · Lambda · Containers · Elastic Beanstalk

Module 6 Objectives

- Identify the different types of AWS compute services
- Explain Amazon EC2 instance types, pricing, and use cases
- Describe Amazon EC2 Auto Scaling and its benefits
- Describe Elastic Load Balancing and how it distributes traffic
- Understand AWS Lambda and serverless computing concepts
- Identify container services: Amazon ECS, EKS, and AWS Fargate
- Describe AWS Elastic Beanstalk for easy app deployment

AWS Compute Services

Virtual Machines

Amazon EC2

Resizable compute in the cloud

Serverless

AWS Lambda

Run code without managing servers

Containers

ECS / EKS / Fargate

Container management & orchestration

PaaS

Elastic Beanstalk

Deploy & scale web apps easily

Auto Scaling

EC2 Auto Scaling

Automatically adjust capacity

Load Balancing

Elastic Load Balancing

Distribute incoming traffic

Amazon EC2 – Elastic Compute Cloud

What is Amazon EC2?

Amazon Elastic Compute Cloud (Amazon EC2) provides resizable compute capacity in the cloud as virtual machines (instances). You have complete control over the instances and can run them for as long as needed.

- Launch instances in minutes
- Choose CPU, memory, storage & networking
- Run on AWS's secure global infrastructure
- Pay only for what you use

Complete Control

Root/admin access, OS customization

Flexible Config

vCPU, RAM, storage, networking

Secure

VPC, security groups, key pairs

Scalable

Scale up/down in minutes

EC2 Instance Types

General Purpose

Examples: t3, m6i

Balanced CPU, memory & networking. Ideal for web servers, dev environments, small databases.

Compute Optimized

Examples: c6i, c7g

High-performance processors. Best for batch processing, HPC, gaming servers, video encoding.

Memory Optimized

Examples: r6i, x2idn

Large memory for in-memory databases. Use for SAP HANA, Redis, Memcached, big data analytics.

Storage Optimized

Examples: i4i, d3

High sequential read/write access. Best for NoSQL databases, data warehousing, OLTP systems.

Accelerated Computing

Examples: p4d, g5, inf2

GPU/FPGA instances. Designed for ML training, inference, graphics-intensive, HPC workloads.

Amazon Machine Images (AMI)

What is an AMI?

An AMI is a template that contains the software configuration (OS, application server, applications) required to launch an instance.

AMI Sources:

- AWS Quick Start AMIs (pre-built)
- AWS Marketplace (third-party)
- Community AMIs (public)
- Custom AMIs (your own)

Root Volume Template

OS + pre-installed apps

Launch Permissions

Controls who can use the AMI

Block Device Mapping

Specifies volumes to attach

EC2 Pricing Models

On-Demand

Pay by the second, no commitment

- No upfront cost or long-term commitment
- Ideal for unpredictable workloads
- Best for dev/test environments
- Highest per-hour price

Reserved

1- or 3-year term, up to 72% savings

- Significant discount vs. On-Demand
- All Upfront / Partial / No Upfront
- Standard or Convertible RIs
- Best for steady-state workloads

Spot

Bid on unused capacity, up to 90% off

- Lowest price available
- Can be interrupted with 2-min notice
- Ideal for batch & fault-tolerant jobs
- Not for critical/persistent workloads

Dedicated

Physical server dedicated to you

- Dedicated Host or Dedicated Instance
- BYOL (Bring Your Own License)
- Compliance & regulatory requirements
- Highest cost option

Amazon EC2 Auto Scaling

Automatically add or remove EC2 instances based on demand – ensuring availability and cost efficiency.

Dynamic Scaling

Responds to changing demand in real-time using CloudWatch alarms and scaling policies (target tracking, step, simple).

Predictive Scaling

Uses ML to forecast future demand and proactively scale capacity ahead of anticipated spikes.

Scheduled Scaling

Scale on a defined schedule — useful for predictable traffic patterns (e.g., business hours).

Auto Scaling Group Configuration

Minimum Capacity
Floor – always running instances

Desired Capacity
Target – normal workload instances

Maximum Capacity
Ceiling – upper limit on instances

Launch Template
AMI, instance type, key pair, SG

Elastic Load Balancing (ELB)

Automatically distributes incoming traffic across multiple targets (EC2, Lambda, containers) in one or more Availability Zones.

Application Load Balancer (ALB)

Layer 7 – HTTP/HTTPS

- Content-based routing (path, host, headers)
- WebSocket and HTTP/2 support
- Best for microservices & containers

Network Load Balancer (NLB)

Layer 4 – TCP/UDP/TLS

- Extreme performance (millions of req/sec)
- Ultra-low latency
- Static IP support, best for TCP traffic

Gateway Load Balancer (GWLB)

Layer 3 + 4 – IP

- Deploy & manage virtual appliances
- Firewalls, IDS/IPS, deep packet inspection
- Scales appliances transparently

AWS Lambda – Serverless Computing

What is AWS Lambda?

Run code without provisioning or managing servers. Pay only for the compute time you consume — no charge when code is not running.

- Supports Node.js, Python, Java, Go, Ruby, .NET
- Automatic scaling – from 1 to thousands of requests
- Up to 15 minutes execution time per invocation
- Max 10 GB memory, 512 MB – 10 GB ephemeral storage
- Built-in fault tolerance & high availability

How Lambda Works

1. **Event Trigger**
   API Gateway, S3, DynamoDB, SNS, CloudWatch Events…
2. **Lambda Executes**
   AWS spins up environment, runs your function code
3. **Auto Scales**
   Concurrent executions scale automatically
4. **You Pay Per Use**
   Billed per 1ms of compute, rounded up

Container Services on AWS

Containers | Package code + dependencies into a single unit. Lightweight, consistent across environments, fast startup times.

Amazon ECS

Elastic Container Service

AWS-native container orchestration. Runs Docker containers. Tight integration with AWS services. Two modes: EC2 or Fargate launch type.

Best for:

When you want AWS-managed orchestration

Amazon EKS

Elastic Kubernetes Service

Managed Kubernetes service. Run containerized workloads using Kubernetes. Full K8s API compatibility. Works on EC2 or Fargate.

Best for:

When you need Kubernetes compatibility

AWS Fargate

Serverless Container Engine

Serverless compute for containers. No need to provision or manage EC2 instances. Pay only for resources consumed by containers.

Best for:

When you want no server management

AWS Elastic Beanstalk

Easy App Deployment & Scaling

The fastest and simplest way to deploy web applications on AWS. Upload your code and Elastic Beanstalk automatically handles deployment, capacity provisioning, load balancing, auto scaling, and health monitoring.

Supported Platforms:

- Java · .NET · Node.js · Python
- Ruby · PHP · Go
- Docker containers
- Custom platforms via Packer

EC2 Instances

Provisions the right compute

Load Balancer

Auto creates & configures ELB

Auto Scaling

Scales capacity automatically

Health Monitoring

Dashboard + auto-repair

Software Updates

Managed platform patching

Choosing the Right Compute Service

Service	Best Use Case	Control	Ease
Amazon EC2	Full control of OS, custom hardware	★★★★★	★★☆☆☆
Elastic Beanstalk	Easy web app deployment, PaaS	★★★☆☆	★★★★★
AWS Lambda	Event-driven, short-lived functions	★★☆☆☆	★★★★★
Amazon ECS/EKS	Containerized microservices	★★★★☆	★★★☆☆
AWS Fargate	Containers without server management	★★★☆☆	★★★★☆

Key Takeaways – Module 6

01 Amazon EC2 provides flexible virtual machines with complete control over OS, instance type, and configuration.

02 EC2 pricing includes On-Demand, Reserved, Spot, and Dedicated options — choose based on workload characteristics.

03 Auto Scaling and Elastic Load Balancing work together to ensure high availability and cost efficiency.

04 AWS Lambda enables serverless, event-driven computing — no server management, pay only per invocation.

05 ECS, EKS, and Fargate provide container orchestration options from self-managed to fully serverless.

06 Elastic Beanstalk simplifies deployment by managing infrastructure so you focus on your application code.